

```
import React, { useEffect, useState } from "react";
import ReactDOM from "react-dom/client";
import { FluentProvider, webLightTheme, webDarkTheme } from "@fluentui/react-components";
import App from "./App";
import { useThemeStore } from "./stores/themeStore";
import { activationService } from "./services/activationService";
import { useStartupFlowStore } from "./stores/startupFlowStore";
import { usePrivacyConsentStore, shouldShowPrivacyConsent } from "./stores/privacyConsentStore";
import "./styles/global.css";
import "./styles/startup-animations.css";

//
import "./utils/securityProtection";
import "./utils/devtools";

//          localStorage          token
localStorage.removeItem('rom-download-storage');
console.log('          localStorage          rom-download-storage');

function AppWithTheme() {
  const { isDarkMode, followSystemTheme, updateThemeBasedOnSystem, subscribeToStorageChanges } =
    useThemeStore();
  const [isActivationValid, setIsActivationValid] = useState(true);
  const [hasAcceptedTerms, setHasAcceptedTerms] = useState(true);
  const { setCurrentPhase } = useStartupFlowStore();
  const { hasCompletedPrivacySetup, hasAcceptedPrivacyPolicy, hasAcceptedUserAgreement } =
    usePrivacyConsentStore();

  //
  useEffect(() => {
    if (followSystemTheme) {
      //
      updateThemeBasedOnSystem();

      //
      const mediaQuery = window.matchMedia('(prefers-color-scheme: dark)');
      const handler = () => updateThemeBasedOnSystem();

      mediaQuery.addEventListener('change', handler);

      return () => mediaQuery.removeEventListener('change', handler);
    }
  }, [followSystemTheme, updateThemeBasedOnSystem]);

  //
  useEffect(() => {
    const cleanup = subscribeToStorageChanges();
    return cleanup;
  }, [subscribeToStorageChanges]);

  //          1
  useEffect(() => {
    const checkActivationStatus = () => {
      try {
        const activationStatus = activationService.checkActivationStatus();

        //
        if (activationStatus.isExpired || activationStatus.needsActivation) {
```

```

        console.log('                    :', activationStatus.expiredReason || ' ');
        setIsActivationValid(false);

        //
        setCurrentPhase('activation-verification');

        //
        if (activationStatus.isExpired) {
            activationService.handleExpiredActivation();
        }
    } else {
        setIsActivationValid(true);
    }
} catch (error) {
    console.error('                    :', error);
    //
    setIsActivationValid(false);
    setCurrentPhase('activation-verification');
}
};

//
checkActivationStatus();

//
1
const intervalId = setInterval(checkActivationStatus, 1000);

//
return () => clearInterval(intervalId);
}, [setCurrentPhase]);

//
useEffect(() => {
    const checkTermsAcceptance = () => {
        try {
            //
            const needsToShowConsent = shouldShowPrivacyConsent();

            //
            const hasAcceptedAllTerms = hasCompletedPrivacySetup &&
                hasAcceptedPrivacyPolicy &&
                hasAcceptedUserAgreement;

            console.log('                    :', {
                needsToShowConsent,
                hasCompletedPrivacySetup,
                hasAcceptedPrivacyPolicy,
                hasAcceptedUserAgreement,
                hasAcceptedAllTerms
            });

            if (needsToShowConsent || !hasAcceptedAllTerms) {
                if (hasAcceptedTerms !== false) { //
                    console.log('                    :');
                    setHasAcceptedTerms(false);
                }
            }
            //
            setCurrentPhase('privacy-consent');

```

```

    } else {
      if (hasAcceptedTerms !== true) { //
        console.log('          :          ');
        setHasAcceptedTerms(true);
      }
    }
  } catch (error) {
    console.error('          :', error);
    //
    if (hasAcceptedTerms !== false) {
      console.log('          :          ');
      setHasAcceptedTerms(false);
    }
    setCurrentPhase('privacy-consent');
  }
};

//
checkTermsAcceptance();

//
//
}, [setCurrentPhase, hasCompletedPrivacySetup, hasAcceptedPrivacyPolicy, hasAcceptedUserAgreement,
hasAcceptedTerms]);

//          useEffect          hasAcceptedTerms
useEffect(() => {
  console.log('          : hasAcceptedTerms =', hasAcceptedTerms);
  //
}, [hasAcceptedTerms]);

//
if (!isActivationValid) {
  //
  //          App          useStartupFlowStore
  return (
    <FluentProvider theme={isDarkMode ? webDarkTheme : webLightTheme}>
      <App />
    </FluentProvider>
  );
}

//
if (!hasAcceptedTerms) {
  //
  //          App          useStartupFlowStore
  return (
    <FluentProvider theme={isDarkMode ? webDarkTheme : webLightTheme}>
      <App />
    </FluentProvider>
  );
}

return (
  <FluentProvider theme={isDarkMode ? webDarkTheme : webLightTheme}>
    <App />
  </FluentProvider>
);

```

```
}

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <AppWithTheme />
  </React.StrictMode>
);

/**
//
//
// 1.
// -
// -
// -      ADB
// -
// 2.
// -
// -
// -
// -
// 3.
// -
// -
//
// */

import TitleBar from "./components/Bar/TitleBar";
import MainContent from "./components/MainContent/MainContent";
import StatusBar from "./components/Bar/StatusBar";
import { ErrorNotification } from "./components/Common/ErrorNotification";
import { StartupFlow } from "./components/StartupFlow/StartupFlow";
import { useAppStyles } from "./styles/appStyles";
import { useAppStartup } from "./hooks/useAppStartup";

function App() {
  const styles = useAppStyles();
  const {
    isLoading,
    error,
    showStartupFlow,
    showErrorNotification,
    countdown,
    showTransition,
    currentPhase,
    handlePrivacyConsent,
    handleActivationSuccess,
    handleDataCollectionConsent,
    handleStartupFlowError,
    handleStartupFlowComplete
  } = useAppStartup();

  //
  if (isLoading) {
    return null;
  }
}
```

```

    }

    //
    if (showErrorNotification && error) {
      return <ErrorNotification error={error} countdown={countdown} />;
    }

    //
    if (showStartupFlow) {
      return (
        <StartupFlow
          currentPhase={currentPhase}
          showTransition={showTransition}
          onPrivacyConsent={handlePrivacyConsent}
          onActivationSuccess={handleActivationSuccess}
          onDataCollectionConsent={handleDataCollectionConsent}
          onStartupFlowError={handleStartupFlowError}
          onStartupFlowComplete={handleStartupFlowComplete}
          onTransitionComplete={() => {
            // useAppStartup Hook
          }}
        />
      );
    }

    //
    return (
      <div className={styles.app}>
        <TitleBar />
        <MainContent />
        <StatusBar />
      </div>
    );
  }
}

```

```
export default App;
```

```

import React, { useEffect, useRef, useState } from 'react';
import {
  makeStyles,
  TabList,
  Tab,
  SelectTabEvent,
  SelectTabData,
  Text,
  Badge,
  Dialog,
  DialogSurface,
  DialogBody,
  DialogTitle,
  DialogContent,
  DialogActions,
  Button,
} from "@fluentui/react-components";
import {
  Code24Regular,

```

```
Settings24Regular,  
CloudArrowUp24Regular,  
CloudArrowDown24Regular,  
Home24Regular,  
Tent24Regular,  
DocumentText24Regular,  
ChevronDown24Regular,  
Warning24Regular,  
} from "@fluentui/react-icons";  
import DeviceSelectionDialog from './DeviceSelectionDialog';  
import { getDeviceIcon } from "../../assets/icons";  
import UnlinkIcon from "../../assets/icons/devices/unlink.gif";  
import { useAppStore } from "../../stores/appStore";  
import { useDeviceStore } from "../../stores/deviceStore";  
import { useDeviceService } from "../../services/deviceService";  
import { AppView } from "../../types/app";  
import HomePage from "../Home/HomePage";  
import AdbZonePanel from "../AdbTools/AdbZonePanel";  
import FlashZonePanel from "../FlashZone/FlashZonePanel";  
import ExtendedFeaturesPanel from "../ExtendedFeatures/ExtendedFeaturesPanel";  
import OnlineZonePanel from "../OnlineResources/OnlineZonePanel";  
import SettingsPanel from "../Settings/SettingsPanel";  
import CarouselComponent from "../CarouselComponent";  
import VersionChecker from "../Common/VersionChecker";  
import RootPanel from "../Root/RootPanel";  
import CommandExecutePanel from '../Others/CommandExecutePanel';  
import LogsPanel from '../Others/LogsPanel';  
  
import { usageTrackingService } from "../../services/usageTrackingService";  
import { systemTrayManager } from "../../services/systemTrayManager";
```

```
const useStyles = makeStyles({  
  '@keyframes pulse': {  
    '&from': {  
      opacity: "1",  
    },  
    '&50%': {  
      opacity: "0.8",  
    },  
    '&to': {  
      opacity: "1",  
    },  
  },  
  container: {  
    flex: 1,  
    display: "flex",  
    flexDirection: "row",  
    overflow: "hidden",  
    height: "100vh",  
    backgroundColor: "var(--colorNeutralBackground1)",  
    gap: "1px", //  
  },  
  sidebar: {  
    width: "180px", //  
    maxWidth: "250px",  
    backgroundColor: "var(--colorNeutralBackground2)", //  
    borderRight: "1px solid var(--colorNeutralStroke3)", //
```

```
    display: "flex",
    flexDirection: "column",
    overflow: "hidden",
    boxShadow: "2px 0 16px rgba(0, 0, 0, 0.08)", //
    position: "relative",
    zIndex: 10,
    //
    background: "linear-gradient(180deg, var(--colorNeutralBackground2) 0%, var(--colorNeutralBackground1) 100%)",
  },
  deviceInfo: {
    padding: "5px", //
    borderBottom: "1px solid var(--colorNeutralStroke3)", //
    backdropFilter: "blur(10px)", //
    overflow: "hidden",
    minHeight: "auto",
    position: "relative",
    boxSizing: "border-box",
    margin: "8px", //
  },
  deviceInfoContainer: {
    display: "flex",
    flexDirection: "column",
    gap: "2px", //
    width: "100%",
    overflow: "hidden",
    boxSizing: "border-box",
  },
  deviceInfoTop: {
    display: "flex",
    alignItems: "flex-start",
    gap: "2px", //
    width: "100%",
    overflow: "hidden",
    marginBottom: "4px", //
    border: "1px solid var(--colorNeutralStroke2)", //
    borderRadius: "8px", //
    padding: "4px", //
  },
  deviceSelectorArea: {
    width: "100%",
    marginTop: "2px",
  },
  deviceSelectCard: {
    width: "100%",
    padding: "8px",
    border: "1px solid var(--colorNeutralStroke2)",
    borderRadius: "8px",
    backgroundColor: "var(--colorNeutralBackground1)",
    cursor: "pointer",
    transition: "all 0.2s ease",
    display: "flex",
    alignItems: "center",
    justifyContent: "space-between",
    '&:hover': {
      backgroundColor: "var(--colorNeutralBackground1Hover)",
      border: "1px solid var(--colorNeutralStroke1)",
    },
  },
```

```
    },
    deviceSelectCardContent: {
      display: "flex",
      alignItems: "center",
      gap: "4px",
      flex: 1,
    },
    deviceSelectCardText: {
      display: "flex",
      flexDirection: "column",
      gap: "2px",
    },
    deviceSelectCardTitle: {
      fontSize: "12px",
      fontWeight: "600",
      color: "var(--colorNeutralForeground1)",
    },
    deviceSelectCardSubtitle: {
      fontSize: "10px",
      color: "var(--colorNeutralForeground3)",
    },
    noDeviceCard: {
      width: "100%",
      padding: "12px",
      border: "1px solid var(--colorNeutralStroke3)",
      borderRadius: "8px",
      backgroundColor: "var(--colorNeutralBackground3)",
      display: "flex",
      alignItems: "center",
      justifyContent: "center",
      gap: "8px",
      cursor: "not-allowed",
    },
    noDeviceCardText: {
      fontSize: "14px",
      color: "var(--colorNeutralForeground3)",
    },
    deviceIconSection: {
      display: "flex",
      alignItems: "center",
      justifyContent: "center",
      width: "40px",
      height: "auto", //
      minHeight: "100px", //
      flexShrink: 0,
      borderRadius: "6px",
      backgroundColor: "var(--colorNeutralBackground2)",
      position: "relative",
      overflow: "hidden",
      boxSizing: "border-box",
      marginLeft: "4px", //
    },
    deviceSerialSection: {
      display: "flex",
      flexDirection: "column",
      gap: "2px",
    },
    deviceSerial: {
```



```
    fontSize: "10px",
    color: "var(--colorNeutralForeground3)",
    fontFamily: "monospace",
  },
  deviceSelector: {
    flex: 1,
    minWidth: 0,
    overflow: "hidden",
  },
  deviceSelectorWrapper: {
    width: "100%",
  },
  deviceSelectDropdown: {
    width: "100%",
    maxWidth: "100%",
    minWidth: "100%",
    overflow: "hidden",
    display: "flex",
    flexDirection: "column",
    alignItems: "flex-start",
    justifyContent: "flex-start",
    gap: "4px",
    minHeight: "20px",
    textAlign: "center",
    padding: "4px",
    boxSizing: "border-box",
    borderRadius: "8px",
    border: "1px solid var(--colorNeutralStroke2)",
    color: "var(--colorNeutralForeground1)",
    backgroundColor: "var(--colorNeutralBackground1)",

  },
  deviceTextInfo: {
    marginTop: "8px",
    marginLeft: "12px",
    flex: 1,
    display: "flex",
    flexDirection: "column",
    gap: "2px", //
    overflow: "hidden",
    minWidth: 0,
    padding: "0", //
    boxSizing: "border-box",
  },
  deviceNameSection: {
    marginBottom: "4px",
  },
  deviceName: {
    fontSize: "16px", //
    fontWeight: "600",
    color: "var(--colorNeutralForeground1)",
    lineHeight: "1.2",
    maxWidth: "100%",
    wordBreak: "break-word",
    overflowWrap: "break-word",
    display: "-webkit-box",
    WebkitLineClamp: 2,
    WebkitBoxOrient: "vertical",
```

```
        overflow: "hidden",
        marginBottom: "4px", //
    },
    deviceCodename: {
        fontSize: "12px", //
        color: "var(--colorNeutralForeground3)",
        lineHeight: "1.1",
        maxWidth: "100%",
        wordBreak: "break-word",
        overflowWrap: "break-word",
        display: "-webkit-box",
        WebkitLineClamp: 1,
        WebkitBoxOrient: "vertical",
        overflow: "hidden",
        fontFamily: "monospace",
        marginTop: "4px", //
    },
    deviceStatusSection: {
        display: "flex",
        flexDirection: "column",
        gap: "4px",
    },
    statusBadgeRow: {
        display: "flex",
        alignItems: "center",
        gap: "4px",
        minWidth: 0,
        marginTop: "4px", //
    },
    statusLabel: {
        fontSize: "9px",
        color: "var(--colorNeutralForeground2)",
        minWidth: "24px",
        flexShrink: 0,
    },
    compactBadge: {
        fontSize: "10px", //
        padding: "3px 6px", //
        minHeight: "18px", //
        borderRadius: "8px", //
        fontWeight: "500", //
        boxShadow: "0 1px 3px rgba(0, 0, 0, 0.1)", //
        width: "80px", //
    },
    deviceInfoHeader: {
        display: "flex",
        alignItems: "flex-start",
        gap: "12px",
        marginBottom: "0",
    },
    deviceIcon: {
        color: "var(--colorBrandForeground1)",
    },
    deviceIconImage: {
        width: "95px", //
        height: "100%", //
        maxHeight: "88px", //
```

```
borderRadius: "4px",
objectFit: "contain", //
flexShrink: 0,
},

deviceTitle: {
  fontSize: "13px",
  fontWeight: "600",
  color: "var(--colorNeutralForeground1)",
  lineHeight: "1.2",
},
deviceDetails: {
  display: "flex",
  flexDirection: "column",
  gap: "4px",
},
deviceRow: {
  display: "flex",
  alignItems: "center",
  justifyContent: "space-between",
  gap: "8px",
  minWidth: 0, //
  overflow: "hidden",
},
deviceLabel: {
  fontSize: "11px",
  color: "var(--colorNeutralForeground3)",
  minWidth: "40px",
  flexShrink: 0, //
},
deviceValue: {
  fontSize: "11px",
  color: "var(--colorNeutralForeground2)",
  fontWeight: "500",
  flex: 1,
  textAlign: "right",
  overflow: "hidden",
  textOverflow: "ellipsis",
  whiteSpace: "nowrap",
  minWidth: 0, //
  maxWidth: "120px", //
},
statusIndicator: {
  display: "flex",
  alignItems: "center",
  gap: "4px",
},
noDevice: {
  padding: "12px",
  borderBottom: "1px solid var(--colorNeutralStroke3)",
  backgroundColor: "var(--colorNeutralBackground2)",
  textAlign: "center",
},
noDeviceIcon: {
  color: "var(--colorNeutralForeground3)",
  marginBottom: "4px",
},
noDeviceText: {
```

```
    fontSize: "11px",
    color: "var(--colorNeutralForeground3)",
  },
//
noDeviceBackgroundImage: {
  width: "80px",
  height: "80px",
  objectFit: "contain",
  opacity: 0.6,
},
noDeviceMessageSection: {
  display: "flex",
  flexDirection: "column",
  gap: "4px",
  justifyContent: "center",
  height: "100%",
  minHeight: "80px", //
},
noDeviceTitle: {
  fontSize: "14px",
  fontWeight: "600",
  color: "var(--colorNeutralForeground2)",
  lineHeight: "1.2",
},
noDeviceSubtitle: {
  fontSize: "11px",
  color: "var(--colorNeutralForeground3)",
  lineHeight: "1.3",
  wordBreak: "break-word",
},
sidebarHeader: {
  padding: "12px 16px 10px",
  borderBottom: "1px solid var(--colorNeutralStroke2)",
  marginBottom: "8px",
  position: "relative",
  backgroundColor: "var(--colorNeutralBackground1)",
  boxSizing: "border-box",
  boxShadow: "0 1px 2px rgba(0, 0, 0, 0.03)",

  // Add a subtle gradient overlay
  "&::after": {
    content: "",
    position: "absolute",
    bottom: "-8px",
    left: 0,
    right: 0,
    height: "8px",
    background: "linear-gradient(to bottom, var(--colorNeutralBackground1), transparent)",
    pointerEvents: "none",
    zIndex: 1,
  }
},
tabList: {
  backgroundColor: "transparent",
  padding: "8px 10px 16px",
  display: "flex",
  flexDirection: "column",
  alignItems: "center",
```

```
gap: "8px",
flex: 1,
overflowY: "auto",
overflowX: "hidden",
boxSizing: "border-box",
scrollBehavior: "smooth",
scrollPaddingTop: "8px",
scrollPaddingBottom: "16px",

// Enhanced scrollbar styling
"&::-webkit-scrollbar": {
  width: "4px",
},
"&::-webkit-scrollbar-track": {
  background: "var(--colorNeutralBackground1)",
  borderRadius: "4px",
},
"&::-webkit-scrollbar-thumb": {
  background: "var(--colorNeutralStroke2)",
  borderRadius: "4px",
  "&:hover": {
    background: "var(--colorNeutralStroke1)",
  },
},
},
tab: {
  width: "100%",
  justifyContent: "flex-start",
  padding: "10px 12px", // Increased padding for better touch targets
  borderRadius: "8px",
  minHeight: "42px", // Increased height for better visibility
  maxWidth: "92%",
  fontSize: "12px",
  fontWeight: "500",
  transition: "all 0.2s cubic-bezier(0.4, 0, 0.2, 1)", // Smoother transition
  position: "relative",
  backgroundColor: "var(--colorNeutralBackground1)",
  color: "var(--colorNeutralForeground2)",
  cursor: "pointer",
  overflow: "hidden",
  boxSizing: "border-box",
  border: "1px solid var(--colorNeutralStroke2)",
  boxShadow: "0 1px 2px rgba(0, 0, 0, 0.02)", // Subtle shadow for depth

  // Icon styles
  "& .fui-Tab__icon": {
    color: "var(--colorNeutralForeground3)",
    fontSize: "16px", // Slightly larger icons
    marginRight: "8px",
    transition: "all 0.2s ease",
    flexShrink: 0,
  },
},

// Hover state
"&:hover": {
  backgroundColor: "var(--colorNeutralBackground2)",
  color: "var(--colorNeutralForeground1)",
  transform: "translateY(-1px)",
```

```
    boxShadow: "0 2px 4px rgba(0, 0, 0, 0.05)",

    "& .fui-Tab__icon": {
      color: "var(--colorBrandForeground1)",
      transform: "scale(1.05)",
    },
  },

  // Selected state
  "&[aria-selected='true']": {
    backgroundColor: "var(--colorBrandBackground2)",
    color: "var(--colorBrandForeground1)",
    border: "1px solid var(--colorBrandStroke2)",
    fontWeight: "600",
    boxShadow: "0 1px 3px rgba(0, 0, 0, 0.05)",
    paddingLeft: "16px", // Indent selected tab

    "& .fui-Tab__icon": {
      color: "var(--colorBrandForeground1)",
      transform: "scale(1.1)",
    },

    // Left indicator for selected tab
    "&::before": {
      content: "''",
      position: "absolute",
      left: "0",
      top: "50%",
      transform: "translateY(-50%)",
      width: "4px",
      height: "70%",
      backgroundColor: "var(--colorBrandForeground1)",
      borderRadius: "0 3px 3px 0",
      transition: "all 0.3s ease",
    },

    // Add subtle pulse animation on selection
    "@media (prefers-reduced-motion: no-preference)": {
      animation: "pulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite",
    },
  },

  //
  "&:active": {
    transform: "scale(0.98)",
  },
},
carouselContainer: {
  padding: "8px", //
  marginTop: "auto",
  borderTop: "1px solid var(--colorNeutralStroke2)", //
  backgroundColor: "var(--colorNeutralBackground1)",
  position: "relative",
  boxSizing: "border-box",
},
buttonGroupContainer: {
  display: "flex",
  justifyContent: "space-between",
```

```
    gap: "8px",
    padding: "8px",
    maxHeight: "50px",
    borderTop: "1px solid var(--colorNeutralStroke2)",
    backgroundColor: "var(--colorNeutralBackground1)",
    boxSizing: "border-box",
  },
  actionBar: {
    flex: 1,
    height: "34px",
    fontSize: "12px",
    fontWeight: "500",
    borderRadius: "6px",
    display: "flex",
    alignItems: "center",
    justifyContent: "center",
    cursor: "pointer",
    transition: "all 0.2s ease",
    border: "1px solid var(--colorNeutralStroke2)",
    backgroundColor: "var(--colorNeutralBackground2)",
    color: "var(--colorNeutralForeground2)",
    gap: "6px", //

    "&:hover": {
      backgroundColor: "var(--colorBrandBackground2)",
      color: "var(--colorBrandForeground1)",
    },

    "&:active": {
      transform: "scale(0.98)",
    },
  },
  actionBarSelected: {
    backgroundColor: "var(--colorBrandBackground1)",
    color: "var(--colorBrandForeground1)",
    border: "1px solid var(--colorBrandStroke1)",
  },
  content: {
    flex: 1,
    overflow: "hidden",
    backgroundColor: "var(--colorNeutralBackground2)",
    position: "relative",

    //
    "&::before": {
      content: "",
      position: "absolute",
      top: 0,
      left: 0,
      right: 0,
      bottom: 0,
      background: "radial-gradient(circle at 50% 50%, var(--colorNeutralBackground1) 0%, transparent 70%)",
      opacity: 0.3,
      pointerEvents: "none",
    },
  },
});
```

```
const tabs = [
  {
    id: "home" as AppView,
    label: "      ",
    icon: <Home24Regular />,
  },
  {
    id: "adb-zone" as AppView,
    label: "      ",
    icon: <Code24Regular />,
  },
  {
    id: "flash-zone" as AppView,
    label: "      ",
    icon: <CloudArrowUp24Regular />,
  },
  {
    id: "online-resources" as AppView,
    label: "      ",
    icon: <CloudArrowDown24Regular />,
  },
  {
    id: "settings" as AppView,
    label: "      ",
    icon: <Settings24Regular />,
  },
];

const MainContent: React.FC = () => {
  const styles = useStyles();
  const { currentView, setCurrentView, config, setStatusBarMessage } = useAppStore();
  const { selectedDevice, devices, selectDevice } = useDeviceStore();
  const { startScanning, stopScanning, refreshDeviceInfo } = useDeviceService();
  const prevConnectedCount = useRef<number>(0);

  //
  const [triggerVersionCheck, setTriggerVersionCheck] = useState(false);
  const [updateCheckCompleted, setUpdateCheckCompleted] = useState(false);

  //
  const [isDeviceSelectionDialogOpen, setIsDeviceSelectionDialogOpen] = useState(false);

  //
  const [isOfflineDialogOpen, setIsOfflineDialogOpen] = useState(false);

  //
  const [isUnauthorizedDialogOpen, setIsUnauthorizedDialogOpen] = useState(false);

  //
  -
  useEffect(() => {
    if (config.autoDetectDevices) {
      //
      stopScanning();
      startScanning();
    } else {
      stopScanning();
    }
  });
}
```



```

    }
    return () => stopScanning();
  }, [config.autoDetectDevices, config.scanInterval, startScanning, stopScanning]);

//
useEffect(() => {
  if (selectedDevice) {
    if (selectedDevice.mode === "offline") {
      setIsOfflineDialogOpen(true);
      setStatusBarMessage({
        type: "warning",
        message: "                ",
      });
    } else if (selectedDevice.mode === "unauthorized") {
      setIsUnauthorizedDialogOpen(true);
      setStatusBarMessage({
        type: "warning",
        message: "                ",
      });
    }
  }
}, [selectedDevice]);

//      /
useEffect(() => {
  const connectedCount = devices.filter(d => d.connected).length;
  const prev = prevConnectedCount.current;

  if (prev === 0 && connectedCount > 0) {
    //
    setStatusBarMessage({
      type: "success",
      message: connectedCount === 1 ? "                1                " : `                ${connectedCount}                `,
    });
  } else if (prev > 0 && connectedCount === 0) {
    //
    setStatusBarMessage({
      type: "warning",
      message: "                ",
    });
  } else if (connectedCount < prev && connectedCount > 0) {
    //
    setStatusBarMessage({
      type: "warning",
      message: "                ",
    });
  } else if (connectedCount > prev && prev > 0) {
    //
    setStatusBarMessage({
      type: "success",
      message: "                ",
    });
  }

  prevConnectedCount.current = connectedCount;
}, [devices]);

//

```

```

useEffect(() => {
  if (selectedDevice && selectedDevice.connected && !selectedDevice.properties) {
    setStatusBarMessage({ type: "info", message: "          ..." });
    refreshDeviceInfo(selectedDevice.serial);
  }
}, [selectedDevice, refreshDeviceInfo]);

//
useEffect(() => {
  if (selectedDevice?.properties) {
    console.log('          : ', {
      marketName: selectedDevice.properties.marketName,
      model: selectedDevice.properties.model,
      brand: selectedDevice.properties.brand,
      manufacturer: selectedDevice.properties.manufacturer,
      deviceName: selectedDevice.properties.deviceName,
      serial: selectedDevice.serial
    });
    setStatusBarMessage({ type: "success", message: "          " });
  }
}, [selectedDevice?.properties]);

//          - MainContent
useEffect(() => {
  const trackMainContentEntry = async () => {
    try {
      console.log(' MainContent          ...');
      console.log('          : ', new Date().toISOString());
      console.log('          : MainContent.tsx useEffect');

      //          HomePage
      setTimeout(async () => {
        try {
          await usageTrackingService.trackMainPageEntry();
          console.log(' MainContent          ');
        } catch (error) {
          console.error(' MainContent          : ', error);
        }
      }, 2000); // 2

    } catch (error) {
      console.error(' MainContent          : ', error);
    }
  };

  console.log(' MainContent useEffect          ');
  trackMainContentEntry();
}, []); //

//          -
useEffect(() => {
  const initializeSystemTray = async () => {
    try {
      //
      if (!systemTrayManager.isReady() && !systemTrayManager.isInitializingNow()) {
        await systemTrayManager.initialize({
          systemTrayEnabled: config.systemTrayEnabled,
          minimizeToTrayOnClose: config.minimizeToTrayOnClose

```

```
    });
  }
} catch (error) {
  console.error('                :', error);
}
};

initializeSystemTray();

//
return () => {
  //
  //
};
}, []); //

//
useEffect(() => {
  const updateTrayConfig = async () => {
    try {
      await systemTrayManager.updateConfig({
        systemTrayEnabled: config.systemTrayEnabled,
        minimizeToTrayOnClose: config.minimizeToTrayOnClose
      });
    } catch (error) {
      console.error('                :', error);
    }
  };
}, [config.systemTrayEnabled, config.minimizeToTrayOnClose]);

//
useEffect(() => {
  //
  const timer = setTimeout(() => {
    if (!updateCheckCompleted) {
      console.log('                ...');
      setTriggerVersionCheck(true);
    }
  }, 10);

  return () => clearTimeout(timer);
}, [updateCheckCompleted]);

//
const handleUpdateCheckComplete = () => {
  console.log('                ');
  setUpdateCheckCompleted(true);
  setTriggerVersionCheck(false);
};

const handleTabSelect = (_event: SelectTabEvent, data: SelectTabData) => {
  setCurrentView(data.value as AppView);
};
```

```
const handleDeviceSelect = (device: any) => {
  selectDevice(device);
};
```

```
const getDeviceMode = () => {
  if (!selectedDevice) return "";

  switch (selectedDevice.mode) {
    case "sys": return " ";
    case "rec": return "Recovery";
    case "fastboot": return "Fastboot";
    case "fastbootd": return "Fastbootd";
    case "sideload": return "Sideload";
    case "edl": return "EDL ";
    case "unauthorized": return " ";
    case "offline": return " ";
    default: return " ";
  }
};
```

```
const connectedDevices = devices.filter(d => d.connected);
```

```
const renderDeviceInfo = () => {
  if (connectedDevices.length === 0) {
    return (
      <div className={styles.deviceInfo}>
        <div className={styles.deviceInfoContainer}>
          {/} + {/}
          <div className={styles.deviceInfoTop}>
            {/} {/}
            <div className={styles.deviceIconSection}>
              <img
                src={UnlinkIcon}
                alt="UnlinkIcon"
                className={styles.deviceIconImage}
              />
            </div>

            {/} {/}
            <div className={styles.deviceTextInfo}>
              <div className={styles.noDeviceMessageSection}>
                <Text className={styles.noDeviceTitle}> </Text>
                <Text className={styles.noDeviceSubtitle}>

                </Text>
              </div>
            </div>
          </div>

          {/} {/}
          <div
            className={styles.deviceSelectCard}
          >
            <div className={styles.deviceSelectCardContent}>
```

```

        <div className={styles.deviceSelectCardTitle}>

            </div>
        </div>
        <Warning24Regular />
    </div>
</div>
);
}

if (!selectedDevice) {
    //
    handleDeviceSelect(connectedDevices[0]);
    return null;
}

const deviceIcon = getDeviceIcon(selectedDevice.mode);

//
//          fastboot          fastboot getvar product
const getDeviceName = () => {
    //          fastboot          product_name
    if (selectedDevice.mode === "fastboot" || selectedDevice.mode === "fastbootd") {
        return selectedDevice.properties?.productName || selectedDevice.serial;
    }
    //
    return selectedDevice.properties?.marketName ||
        selectedDevice.properties?.model ||
        selectedDevice.serial;
};

const getDeviceOptionText = (device: any) => {
    //
    return device.serial;
};

const getDeviceCodeName = () => {
    if (selectedDevice.mode === "fastboot" || selectedDevice.mode === "fastbootd") {
        //          fastboot          product_name
        return selectedDevice.properties?.productName || selectedDevice.serial;
    }
    return selectedDevice.properties?.deviceName || "";
};

return (
    <div className={styles.deviceInfo}>
        <div className={styles.deviceInfoContainer}>
            {/}
            +
            {/}
        <div className={styles.deviceInfoTop}>
            {/}
            {/}
        <div className={styles.deviceIconSection}>
            <img
                src={deviceIcon}
                alt="Device Icon"
                className={styles.deviceIconImage}
            />
        </div>
    </div>

```

```

    {/*          */}
    <div className={styles.deviceTextInfo}>
    {/*          */}
    <div className={styles.deviceNameSection}>
    {/*          */}
    <Text className={styles.deviceName}>
      {getDeviceName()}
    </Text>

    {/*          */}
    {selectedDevice.properties?.deviceName && (
      <Badge
        appearance="outline"
        color="brand"
        size="medium"
        className={styles.compactBadge}
      >
        {getDeviceCodeName()}
      </Badge>
    )}
  </div>

  {/*          */}
  <div className={styles.deviceStatusSection}>
  {/*          */}
  <div className={styles.statusBadgeRow}>
    <Badge
      appearance="outline"
      color="brand"
      size="medium"
      className={styles.compactBadge}
    >
      {getDeviceMode()}
    </Badge>
  </div>
</div>
</div>
</div>

  {/*          */}
  <div className={styles.deviceSelectorArea}>
    {connectedDevices.length > 0 ? (
      <div
        className={styles.deviceSelectCard}
        onClick={() => setIsDeviceSelectionDialogOpen(true)}
      >
        <div className={styles.deviceSelectCardContent}>

          <div className={styles.deviceSelectCardTitle}>
            :
            {getDeviceOptionText(selectedDevice)}
          </div>
        </div>
        <ChevronDown24Regular />
      </div>
    ) : (
      <div
        className={styles.deviceSelectCard}
      >

```

```

        <div className={styles.deviceSelectCardContent}>
            <div className={styles.deviceSelectCardTitle}>

                </div>
            </div>
            <Warning24Regular />
        </div>
    )}
</div>
</div>
</div>
);
};

```

```

const renderContent = () => {
    switch (currentView) {
        case "home":
            return <HomePage />;
        case "adb-zone":
            return <AdbZonePanel />;
        case "flash-zone":
            return <FlashZonePanel />;
        case "root":
            return <RootPanel />;
        case "extended-features":
            return <ExtendedFeaturesPanel />;
        case "online-resources":
            return <OnlineZonePanel />;
        case "settings":
            return <SettingsPanel />;
        case "command-line":
            return <CommandExecutePanel />;
        case "logs":
            return <LogsPanel />;
        default:
            return <HomePage />;
    }
};

```

```

return (
    <div className={` ${styles.container} startup-optimized`}>
        <div className={` ${styles.sidebar} sidebar-enter`}>
            {/*          */}
            {renderDeviceInfo()}

            {/*          */}
            <TabList
                selectedValue={currentView}
                onTabSelect={handleTabSelect}
                className={styles.tabList}
                vertical //
            >
                {tabs.map((tab) => (
                    <Tab
                        key={tab.id}
                        value={tab.id}
                        icon={tab.icon}
                        className={styles.tab}

```

```

    >
    {tab.label}
  </Tab>
))}
</TabList>

```

```

{ /*
*/}
<div className={styles.carouselContainer}>
  <CarouselComponent />
</div>
<div className={styles.buttonGroupContainer}>
  { /*
*/}
  <div
    className={` ${styles.actionButton} ${currentView === "command-line" ?
styles.actionButtonSelected : ""}`}
    onClick={() => setCurrentView("command-line")}
  >
    <Text24Regular />

  </div>
  { /*
*/}
  <div
    className={` ${styles.actionButton} ${currentView === "logs" ? styles.actionButtonSelected :
""}`}
    onClick={() => setCurrentView("logs")}
  >
    <DocumentText24Regular />

  </div>
</div>
</div>

```

```

{ /*
*/}
<DeviceSelectionDialog
  open={isDeviceSelectionDialogOpen}
  onOpenChange={(open) => setIsDeviceSelectionDialogOpen(open)}
  devices={connectedDevices}
  selectedDevice={selectedDevice}
  onDeviceSelect={(device) => {
    handleDeviceSelect(device);
    setIsDeviceSelectionDialogOpen(false);
  }}
/>

```

```

<div className={` ${styles.content} main-content-enter`} >
  {renderContent()}
</div>

```

```

{ /*
-
*/}
<VersionChecker
  triggerCheck={triggerVersionCheck}
  onCheckUpdate={() => console.log('...')}
  onUpdateFound={(result) => {
    console.log('...', result);
    handleUpdateCheckComplete();
  }}
  onNoUpdate={currentVersion => {

```



```

        console.log('                :!', currentVersion);
        handleUpdateCheckComplete();
    }}
    onError={ (error) => {
        console.error('                :!', error);
        handleUpdateCheckComplete();
    }}
    showStatusMessage={false} //
/>

    {/ *                */}
    <Dialog
        onOpenChange={
            (isOpen) => setIsOfflineDialogOpen(!isOpen)
        }
        open={isOfflineDialogOpen}
    >
        <DialogSurface>
            <DialogBody>
                <DialogTitle>                </DialogTitle>
                <DialogContent>
                    <p>                                </p>
                </DialogContent>
                <DialogActions>
                    <Button appearance="secondary" onClick={() => setIsOfflineDialogOpen(false)}>
</Button>
                </DialogActions>
            </DialogBody>
        </DialogSurface>
    </Dialog>

    {/ *                */}
    <Dialog
        onOpenChange={
            (isOpen) => setIsUnauthorizedDialogOpen(!isOpen)
        }
        open={isUnauthorizedDialogOpen}
    >
        <DialogSurface>
            <DialogBody>
                <DialogTitle>                </DialogTitle>
                <DialogContent>
                    <p>                                USB                                </p>
                </DialogContent>
                <DialogActions>
                    <Button appearance="secondary" onClick={() => setIsUnauthorizedDialogOpen(false)}>
</Button>
                </DialogActions>
            </DialogBody>
        </DialogSurface>
    </Dialog>
</div>
);
};

export default MainContent;

```

```
import React from "react";
import {
  makeStyles,
  mergeClasses,
  Text,

  tokens,
} from "@fluentui/react-components";
import {
  Home24Regular,

} from "@fluentui/react-icons";
import { useDeviceStore } from "../../stores/deviceStore";
import { useAppStore } from "../../stores/appStore";

//
import DeviceOverviewCard from "../DeviceInfo/DeviceOverviewCard";
import DeviceRebootCard from "../DeviceRebootCard";
import MiscellaneousCard from "../MiscellaneousCard";

import NoDevicePrompt from "../NoDevicePrompt";

const useStyles = makeStyles({
  container: {
    padding: "16px",
    height: "100%",
    overflow: "auto",
    display: "flex",
    flexDirection: "column",
    gap: "16px",
    position: "relative",
    backgroundColor: "var(--colorNeutralBackground1)",
  },
  backgroundDecoration: {
    position: "absolute",
    top: "0",
    right: "0",
    width: "200px",
    height: "200px",
    background: `linear-gradient(135deg, ${tokens.colorBrandBackground2} 0%, transparent 70%)`,
    borderRadius: "0 0 0 100%",
    opacity: 0.1,
    pointerEvents: "none",
    zIndex: 0,
  },
  header: {
    display: "flex",
    alignItems: "center",
    justifyContent: "space-between",
    flexWrap: "wrap",
    gap: "12px",
    minHeight: "40px",
  },
  headerLeft: {
    display: "flex",
    alignItems: "center",
    gap: "8px",
  },
});
```

```
headerRight: {
  display: "flex",
  alignItems: "center",
  gap: "8px",
},
deviceSelector: {
  minWidth: "180px",
},
mainContent: {
  display: "flex",
  flexDirection: "column",
  gap: "16px",
  flex: 1,
  height: "100%",
},
//
noDeviceFullScreen: {
  height: "100%",
  width: "100%",
  display: "flex",
  alignItems: "center",
  justifyContent: "center",
},
//
deviceSection: {
  display: "flex",
  flexDirection: "column",
  gap: "16px",
},
//
mainContentGrid: {
  display: "flex",
  flexDirection: "column",
  gap: "16px",
  flex: 1,
  height: "100%",
},
//
deviceOverviewSection: {
  flex: "0 0 50%", // 60%
  display: "flex",
  flexDirection: "column",
},
deviceInfoCard: {
  height: "100%",
  overflow: "auto",
},
//
deviceActionsSection: {
  flex: "1 1 50%", // 40%
  display: "flex",
  flexDirection: "row", // 40%
  gap: "26px",
  minHeight: "260px", // 40%
},
//
rebootCard: {
```

```

    flex: "0 0 48%", //
    minHeight: "100%",
    display: "flex",
    flexDirection: "column",
  },
  //
  miscCard: {
    flex: "0 0 48%", //
    minHeight: "100%",
    display: "flex",
    flexDirection: "column",
  },
  noDevice: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    justifyContent: "center",
    gap: "16px",
    padding: "48px 24px",
    textAlign: "center",
    backgroundColor: "var(--colorNeutralBackground2)",
    borderRadius: "8px",
    border: "2px dashed var(--colorNeutralStroke2)",
  },
});

```

```

const HomePage: React.FC = () => {
  const styles = useStyles();
  const {
    devices,
    selectedDevice,
    isScanning
  } = useDeviceStore();
  const { setStatusBarMessage } = useAppStore();

  //
  MainContent

  const connectedDevices = devices.filter(d => d.connected);

  //
  const handleManualRefresh = () => {
    //
    setStatusBarMessage({
      type: "info",
      message: "
      ...",
    });
  };

  return (
    <div className={` ${styles.container} startup-optimized`} >
      { /* */ }
      <div className={styles.backgroundDecoration} />

      { /* */ }

      { /* */ }
      <div className={styles.mainContent}>
        {connectedDevices.length === 0 ? (

```

```

/*                                     */
<div className="card-enter">
  <NoDevicePrompt
    isScanning={isScanning}
    onRefresh={handleManualRefresh}
  />
</div>
):(
/*                                     */
<div className={mergeClasses(styles.deviceSection)}>
  {selectedDevice ? (
    <
      {
        /*                                     */
      }
    <div className={mergeClasses(styles.mainContentGrid)}>
      {
        /*                                     - 60%      */
      }
      <div className={mergeClasses(styles.deviceOverviewSection, "card-enter")}>
        <div className={styles.deviceInfoCard}>
          <DeviceOverviewCard
            device={selectedDevice}
            onShowDetails={() => {
              //
            }}
            onCopyInfo={() => {
              //
              setStatusBarMessage({
                type: "success",
                message: "
",
              });
            }}
            onCustomize={() => {
              //
            }}
          />
        </div>
      </div>

      {
        /*                                     - 40%      */
      }
      <div className={mergeClasses(styles.deviceActionsSection)}>
        {
          /*                                     */
        }
        <div className={mergeClasses(styles.rebootCard, "card-enter-delayed")}>
          <DeviceRebootCard />
        </div>

        {
          /*                                     */
        }
        <div className={mergeClasses(styles.miscCard, "card-enter-delayed")}>
          <MiscellaneousCard />
        </div>
      </div>
    </div>
  ) : (
    <div className={` ${styles.noDevice} card-enter`} >
      <Home24Regular style={{ fontSize: "48px", color: "var(--colorNeutralForeground3)" }} />
      <Text size={400}>
        </Text>
      <Text size={300} style={{ color: "var(--colorNeutralForeground2)" }}>
        </Text>
    </div>
  ) : (

```

```
        </div>
    })
</div>
})
</div>
```

```
</div>
);
};
```

```
export default HomePage;
```